SCITEC LABORATÓRIO

Ferramentaria OS

Documentação técnica completa do sistema de gestão de ordens de serviço

Versão 1.0 · Gerado em abril de 2026 · Responsável: Heber Rosa <heber.rosa@scitec.com.br>

ÍNDICE

1. Visão Geral do Sistema
2. Ferramentas e Tecnologias
    - Stack principal
    - Serviços externos
    - Dependências npm
3. Arquitetura e Estrutura de Pastas
4. Banco de Dados (Supabase / PostgreSQL)
    - Schema SQL
    - Descrição das tabelas
    - Storage (anexos)
    - SQL adicional
5. Variáveis de Ambiente
6. Códigos-fonte
    - types/index.ts
    - lib/constants.ts
    - lib/auth.ts
    - lib/email.ts
    - middleware.ts
    - app/page.tsx (painel público)
    - app/api/pedidos/route.ts
    - app/api/pedidos/[id]/status/route.ts
    - app/api/pedidos/[id]/editar-solicitante/route.ts
    - app/api/auth/login/route.ts
    - app/acompanhar/[token]/page.tsx
    - components/forms/FormularioPedido.tsx
    - components/pedido/EditarPedido.tsx
7. Fluxos principais
8. Deploy e Hospedagem

1. Visão Geral do Sistema

O **Ferramentaria OS** é um sistema web de gestão de ordens de serviço desenvolvido para o setor de Ferramentaria da SCiTec Laboratório. Qualquer colaborador da empresa pode abrir um pedido de serviço sem necessidade de login. A equipe da ferramentaria gerencia os pedidos por meio de um painel Kanban com arrastar e soltar.

URL DE PRODUÇÃO

https://ferramentaria-os.vercel.app

Funcionalidades

- **Painel público (Kanban read-only):** Qualquer pessoa acessa e vê o quadro de pedidos sem login.
- **Abertura de pedido:** Formulário em /solicitar com campos obrigatórios, urgência, prazo desejado e upload de arquivos.
- **Acompanhamento por token:** Link único enviado por e-mail — /acompanhar/[token] — mostra status, histórico e comentários.
- **Edição pelo solicitante:** Na página de acompanhamento, o solicitante pode corrigir descrição, telefone e prazo desejado (enquanto não estiver concluído/cancelado).
- **Painel admin (Kanban interativo):** Login com e-mail/senha. Arrastar cards entre colunas muda o status. Disponível em /admin/dashboard/kanban.
- **Tabela admin:** Visão tabular com filtros, busca e exportação para Excel.
- **Gestão de tipos de serviço:** Admin cria/edita/desativa tipos em /admin/dashboard/tipos-servico.
- **Gestão de usuários:** Admin cria/edita contas de outros usuários (editor/admin).
- **Anexos:** Upload de arquivos na criação do pedido e pelo admin. Armazenados no Supabase Storage.
- **Comentários:** Internos (só admin vê) e públicos (visíveis ao solicitante).
- **E-mails automáticos:** Confirmação ao solicitante, notificação ao admin no novo pedido, alerta de mudança de status (solicitante + admin), alerta de edição pelo solicitante (admin).

2. Ferramentas e Tecnologias

Stack principal

Camada	Tecnologia	Versão	Função
Framework	Next.js (App Router)	14.2	SSR, API Routes, roteamento de páginas
Linguagem	TypeScript	5.x	Tipagem estática em todo o projeto
Estilização	Tailwind CSS	3.4	Utilitários CSS, paleta SCiTec (#C41E3A)
Runtime	Node.js	18+	Execução do servidor Next.js

Serviços externos

Serviço	Uso	Plano
Supabase	Banco PostgreSQL + Storage de arquivos (bucket "anexos")	Free tier
Resend	Envio de e-mails transacionais	Free tier (3.000 e-mails/mês)
Vercel	Hospedagem, deploy automático do GitHub	Hobby (free)
GitHub	Controle de versão, repositório do código	Free, repositório privado

Dependências npm (package.json)

Pacote	Versão	Finalidade
@supabase/supabase-js	^2.45	Cliente Supabase (banco + storage)
resend	^4.0	SDK de envio de e-mails
jose	^5.9	Criação e verificação de JWT (autenticação admin)
bcryptjs	^2.4	Hash de senhas no login admin
zod	^3.23	Validação de dados nas API Routes
@dnd-kit/core + sortable	^6.1 / ^8.0	Drag-and-drop do Kanban admin
xlsx	^0.18	Exportação de pedidos para Excel/CSV
next	14.2	Framework principal
react / react-dom	^18	Interface do usuário
tailwindcss	^3.4	Framework CSS
typescript	^5	Superset JavaScript tipado

3. Arquitetura e Estrutura de Pastas

O projeto segue a convenção do App Router do Next.js 14:

```
ferramentaria-os/
├── app/
│   ├── page.tsx                        ← Painel público (Kanban read-only)
│   ├── solicitar/page.tsx              ← Formulário de abertura de pedido
│   ├── acompanhar/[token]/page.tsx     ← Acompanhamento por token (server)
│   ├── admin/
│   │   ├── page.tsx                    ← Tela de login
│   │   ├── layout.tsx
│   │   └── dashboard/
│   │       ├── layout.tsx
│   │       ├── kanban/page.tsx         ← Kanban interativo (admin)
│   │       ├── tabela/page.tsx         ← Tabela com filtros e exportação
│   │       ├── pedidos/[id]/page.tsx   ← Detalhe do pedido
│   │       ├── usuarios/page.tsx       ← Gestão de usuários
│   │       └── tipos-servico/page.tsx  ← Gestão de tipos de serviço
│   └── api/
│       ├── pedidos/
│       │   ├── route.ts                ← GET (listar) + POST (criar pedido)
│       │   └── [id]/
│       │       ├── route.ts            ← GET + PATCH (update) + DELETE
│       │       ├── status/route.ts     ← PATCH status (requer auth admin)
│       │       ├── comentarios/route.ts ← GET + POST comentários
│       │       ├── anexos/route.ts     ← GET + POST + DELETE anexos
│       │       └── editar-solicitante/route.ts ← PATCH via token público
│       ├── auth/
│       │   ├── login/route.ts          ← POST login admin
│       │   └── logout/route.ts         ← POST logout (clear cookie)
│       ├── export/route.ts             ← GET exportar Excel
│       ├── tipos-servico/
│       │   ├── route.ts                ← GET (público) + POST (admin)
│       │   └── [id]/route.ts           ← PATCH + DELETE
│       └── usuarios/
│           ├── route.ts                ← GET + POST usuários
│           └── [id]/route.ts           ← PATCH + DELETE
├── components/
│   ├── forms/FormularioPedido.tsx      ← Formulário principal (client)
│   ├── kanban/
│   │   ├── KanbanBoard.tsx             ← Board read-only (server)
│   │   ├── KanbanBoardAdmin.tsx        ← Board admin com DnD (client)
│   │   ├── KanbanCard.tsx              ← Card do Kanban
│   │   └── KanbanColumn.tsx            ← Coluna do Kanban
│   ├── pedido/
│   │   ├── Comentarios.tsx             ← Lista + formulário de comentários
│   │   ├── Anexos.tsx                  ← Lista + upload de anexos
│   │   └── EditarPedido.tsx            ← Edição pelo solicitante (client)
│   └── ui/
│       ├── StatusBadge.tsx
│       └── UrgenciaBadge.tsx
├── lib/
│   ├── auth.ts                         ← Funções JWT (criar/verificar token)
│   ├── constants.ts                    ← STATUS_CONFIG, URGENCIA_CONFIG
│   ├── email.ts                        ← Todas as funções de e-mail (Resend)
│   ├── supabase-server.ts              ← Cliente Supabase (service role)
│   └── supabase-browser.ts             ← Cliente Supabase (anon key)
├── types/index.ts                      ← Interfaces TypeScript
├── middleware.ts                       ← Proteção de rotas /admin/dashboard
├── supabase/schema.sql                 ← DDL completo do banco
├── tailwind.config.ts                  ← Paleta de cores SCiTec
├── next.config.mjs
└── package.json
```

4. Banco de Dados

O banco é hospedado no **Supabase** (PostgreSQL gerenciado). A URL do projeto é: `https://yygsawyilwpyqgnyxwho.supabase.co`

Tabelas

Tabela	Descrição
usuarios	Contas da equipe da ferramentaria (admin / editor). Senha armazenada como bcrypt hash.
pedidos	Ordens de serviço. Contém todos os dados do pedido, status atual, responsável e token único de acompanhamento.
historico_status	Cada mudança de status gera um registro aqui. Funciona como trilha de auditoria.
comentarios	Comentários internos (visíveis só ao admin) ou públicos (visíveis ao solicitante na página de acompanhamento).
tipos_servico	Lista de tipos de serviço disponíveis no formulário. Gerenciada pelo admin.
anexos	Metadados dos arquivos enviados (nome, tamanho, tipo MIME, path no Supabase Storage).

Schema SQL completo

supabase/schema.sql

```sql
-- Habilitar extensão para UUID
create extension if not exists "pgcrypto";

-- Tabela de usuários (admin/editores da ferramentaria)
create table if not exists usuarios (
  id uuid primary key default gen_random_uuid(),
  nome text not null,
  email text unique not null,
  senha_hash text not null,
  role text not null default 'editor', -- 'admin' | 'editor'
  ativo boolean default true,
  criado_em timestamptz default now()
);

-- Tabela de pedidos
create table if not exists pedidos (
  id uuid primary key default gen_random_uuid(),
  numero serial,
  setor text not null,
  solicitante text not null,
  email_contato text not null,
  telefone text,
  tipo_servico text not null,
  descricao text not null,
  urgencia text not null default 'normal',
  prazo_desejado date,
  prazo_definido date,
  status text not null default 'recebido',
  responsavel_id uuid references usuarios(id),
  token_acompanhamento text unique default encode(gen_random_bytes(16), 'hex'),
  observacoes_internas text,
  criado_em timestamptz default now(),
  atualizado_em timestamptz default now()
);

-- Tabela de histórico de status
create table if not exists historico_status (
  id uuid primary key default gen_random_uuid(),
  pedido_id uuid references pedidos(id) on delete cascade,
  status_anterior text,
  status_novo text not null,
  observacao text,
  usuario_nome text,
  criado_em timestamptz default now()
);

-- Tabela de comentários
create table if not exists comentarios (
  id uuid primary key default gen_random_uuid(),
  pedido_id uuid references pedidos(id) on delete cascade,
  usuario_nome text not null,
  conteudo text not null,
  interno boolean default false,
  criado_em timestamptz default now()
);

-- Trigger para atualizar atualizado_em
create or replace function update_atualizado_em()
returns trigger as $$
begin
  new.atualizado_em = now();
  return new;
end;
$$ language plpgsql;

create trigger pedidos_atualizado_em
  before update on pedidos
```

```
  for each row execute function update_atualizado_em();

-- RLS (Row Level Security)
alter table pedidos enable row level security;
alter table usuarios enable row level security;
alter table historico_status enable row level security;
alter table comentarios enable row level security;

-- Política permissiva para service role (API Next.js usa service role key)
create policy "Allow service role all" on pedidos for all using (true);
create policy "Allow service role all" on usuarios for all using (true);
create policy "Allow service role all" on historico_status for all using (true);
create policy "Allow service role all" on comentarios for all using (true);
```

SQL adicional (executado posteriormente)

```
Tabela tipos_servico + Tabela anexos + usuário admin inicial
```

```
-- Tipos de serviço (gerenciados pelo admin)
create table if not exists tipos_servico (
  id uuid primary key default gen_random_uuid(),
  nome text not null unique,
  ordem int default 0,
  ativo boolean default true,
  criado_em timestamptz default now()
);

alter table tipos_servico enable row level security;
create policy "Allow service role all" on tipos_servico for all using (true);

-- Popular com tipos iniciais
insert into tipos_servico (nome, ordem) values
  ('Dispositivo de Ensaio', 1),
  ('Preparação de Amostra', 2),
  ('Serralheria', 3),
  ('Metalografia', 4),
  ('Manutenção Predial', 5),
  ('Serviço Terceirizado', 6),
  ('Outros', 7);

-- Tabela de anexos
create table if not exists anexos (
  id uuid primary key default gen_random_uuid(),
  pedido_id uuid references pedidos(id) on delete cascade,
  nome_arquivo text not null,
  tamanho int,
  tipo_mime text,
  storage_path text not null,
  criado_em timestamptz default now()
);

alter table anexos enable row level security;
create policy "Allow service role all" on anexos for all using (true);

-- Storage bucket "anexos" (criar na interface do Supabase → Storage → New bucket)
-- Nome: anexos
-- Tipo: public

-- Usuário admin inicial (senha: trocar após primeiro acesso)
-- Hash bcrypt gerado com: node -e "const b=require('bcryptjs'); console.log(b.hashSync('SUA_SENHA',12))"
insert into usuarios (nome, email, senha_hash, role)
values (
  'Heber Rosa',
  'heber.rosa@scitec.com.br',
  '$2a$12$WEAyQYDoPCkkdTTwkfYEEuK4xhUtXzerRIT8ti6CNxz6cwgyPEDMy', -- trocar
  'admin'
);
```

Supabase Storage (bucket "anexos")

- Criado manualmente em: Supabase → Storage → New bucket → Nome: **anexos** → Marcar como **Public**
- Os arquivos são armazenados no caminho: `{pedido_id}/{timestamp}_{nome_arquivo}`
- O download usa URL pública gerada pelo Supabase Storage
- Limite por arquivo: 50 MB

5. Variáveis de Ambiente

Arquivo `.env.local` (local, não vai para o git). No Vercel, configuradas em *Settings → Environment Variables*.

ATENÇÃO — SEGURANÇA

Nunca commitar o arquivo .env.local. As chaves abaixo são do ambiente de produção atual. Em caso de migração para servidores da empresa, gerar novas chaves e revogar as atuais.

Variável	Onde obter	Descrição
NEXT_PUBLIC_SUPABASE_URL	Supabase → Project Settings → API	URL do projeto Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase → Project Settings → API	Chave pública (anon) para client-side
SUPABASE_SERVICE_ROLE_KEY	Supabase → Project Settings → API	Chave secreta usada nas API Routes (server-side apenas)
JWT_SECRET	Gerar: openssl rand -hex 64	Segredo para assinar/verificar tokens JWT dos admins
RESEND_API_KEY	resend.com → API Keys	Chave para envio de e-mails pelo Resend
EMAIL_FROM	Configurado no Resend	Remetente dos e-mails automáticos
EMAIL_ADMIN	E-mail do responsável	Destinatário das notificações administrativas
NEXT_PUBLIC_BASE_URL	URL do site	Base para gerar links nos e-mails (ex: https://ferramentaria-os.vercel.app)

.env.local (valores de produção atuais)

```
NEXT_PUBLIC_SUPABASE_URL=https://yygsawyilwpyqgnyxwho.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_xNYl0oZxRqfqDb_l4Kaacw_jhzA3TTa
SUPABASE_SERVICE_ROLE_KEY=sb_secret_Rxe96uVb90dM4ETB1YvcAQ_Jg5DU_e4
JWT_SECRET=NnJY8EyFYh6hQW2gNOgr9o4TIlSuPK7bebAnsOtnFh54xHo0HH7HzzlTeoPcVAXD
RESEND_API_KEY=re_JgDwaBXV_JrgZnep2FiDzxzngoAWYWMJN
EMAIL_FROM=onboarding@resend.dev
EMAIL_ADMIN=heber.rosa@scitec.com.br
NEXT_PUBLIC_BASE_URL=https://ferramentaria-os.vercel.app
```

6. Códigos-fonte

types/index.ts — Interfaces TypeScript

types/index.ts

```
export type StatusPedido =
  | 'recebido'
  | 'em_analise'
  | 'aguardando_material'
  | 'em_andamento'
  | 'concluido'
  | 'cancelado'

export type Urgencia = 'normal' | 'alta' | 'urgente'
export type RoleUsuario = 'admin' | 'editor'

export interface Pedido {
  id: string
  numero: number
  setor: string
  solicitante: string
  email_contato: string
  telefone?: string
  tipo_servico: string
  descricao: string
  urgencia: Urgencia
  prazo_desejado?: string
  prazo_definido?: string
  status: StatusPedido
  responsavel_id?: string
  token_acompanhamento: string
  observacoes_internas?: string
  criado_em: string
  atualizado_em: string
  responsavel?: Usuario
}

export interface Usuario {
  id: string
  nome: string
  email: string
  role: RoleUsuario
  ativo: boolean
  criado_em: string
}

export interface HistoricoStatus {
  id: string
  pedido_id: string
  status_anterior?: string
  status_novo: string
  observacao?: string
  usuario_nome?: string
  criado_em: string
}

export interface Comentario {
  id: string
  pedido_id: string
  usuario_nome: string
  conteudo: string
  interno: boolean
  criado_em: string
}

export interface JWTPayload {
  id: string
  email: string
  nome: string
  role: RoleUsuario
}
```

lib/constants.ts — Configurações de status e urgência

lib/constants.ts

```
import { StatusPedido } from '@/types'

export const STATUS_CONFIG: Record<
  StatusPedido,
  { label: string; color: string; bg: string; border: string; dot: string }
> = {
  recebido:            { label: 'Recebido',            color: 'text-blue-700',   bg: 'bg-blue-50',   border: 'bor
der-blue-200',   dot: 'bg-blue-500'   },
  em_analise:          { label: 'Em Análise',           color: 'text-yellow-700', bg: 'bg-yellow-50', border: 'bo
rder-yellow-200', dot: 'bg-yellow-500' },
  aguardando_material: { label: 'Aguardando Material',  color: 'text-orange-700', bg: 'bg-orange-50', border: 'bo
rder-orange-200', dot: 'bg-orange-500' },
  em_andamento:        { label: 'Em Andamento',         color: 'text-purple-700', bg: 'bg-purple-50', border: 'bo
rder-purple-200', dot: 'bg-purple-500' },
  concluido:           { label: 'Concluído',            color: 'text-green-700',  bg: 'bg-green-50',  border: 'bo
rder-green-200',  dot: 'bg-green-500'  },
  cancelado:           { label: 'Cancelado',            color: 'text-gray-600',   bg: 'bg-gray-50',   border: 'bo
rder-gray-200',   dot: 'bg-gray-400'   },
}

export const URGENCIA_CONFIG = {
  normal:  { label: 'Normal',  color: 'text-gray-600',  bg: 'bg-gray-100'  },
  alta:    { label: 'Alta',    color: 'text-yellow-700',bg: 'bg-yellow-100'},
  urgente: { label: 'Urgente', color: 'text-red-700',   bg: 'bg-red-100'   },
}

export const TIPOS_SERVICO = [
  'Dispositivo de Ensaio', 'Preparação de Amostra', 'Serralheria',
  'Metalografia', 'Manutenção Predial', 'Serviço Terceirizado', 'Outros',
]

export const STATUS_ORDER: StatusPedido[] = [
  'recebido', 'em_analise', 'aguardando_material', 'em_andamento', 'concluido', 'cancelado',
]
```

lib/auth.ts — JWT (autenticação admin)

lib/auth.ts

```
import { SignJWT, jwtVerify } from 'jose'
import { JWTPayload } from '@/types'

const getSecret = () => new TextEncoder().encode(process.env.JWT_SECRET!)

export async function criarToken(payload: JWTPayload): Promise<string> {
  return await new SignJWT({ ...payload })
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('8h')
    .sign(getSecret())
}

export async function verificarToken(token: string): Promise<JWTPayload | null> {
  try {
    const { payload } = await jwtVerify(token, getSecret())
    return {
      id: payload.id as string,
      email: payload.email as string,
      nome: payload.nome as string,
      role: payload.role as JWTPayload['role'],
    }
  } catch {
    return null
  }
}
```

middleware.ts — Proteção de rotas admin

middleware.ts

```
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'
import { jwtVerify } from 'jose'

export async function middleware(request: NextRequest) {
  if (request.nextUrl.pathname.startsWith('/admin/dashboard')) {
    const token = request.cookies.get('ferramentaria_token')?.value
    if (!token) {
      return NextResponse.redirect(new URL('/admin', request.url))
    }
    try {
      const secret = new TextEncoder().encode(process.env.JWT_SECRET!)
      await jwtVerify(token, secret)
      return NextResponse.next()
    } catch {
      return NextResponse.redirect(new URL('/admin', request.url))
    }
  }
  return NextResponse.next()
}

export const config = {
  matcher: ['/admin/dashboard/:path*'],
}
```

lib/email.ts — E-mails automáticos (Resend)

Quatro funções de e-mail exportadas:

- **enviarConfirmacaoPedido(pedido)** → para o solicitante ao criar pedido
- **enviarNotificacaoAdmin(pedido)** → para o admin ao criar pedido
- **enviarNotificacaoStatus(pedido, anterior, novo, obs?)** → para o solicitante ao mudar status
- **enviarNotificacaoAdminStatus(pedido, anterior, novo, usuarioNome, obs?)** → para o admin ao mover card
- **enviarNotificacaoAdminEdicao(pedido, camposAlterados[])** → para o admin quando solicitante edita

lib/email.ts (cabeçalho + template base)

```
import { Resend } from 'resend'
import { Pedido, StatusPedido } from '@/types'
import { STATUS_CONFIG } from '@/lib/constants'

const resend = new Resend(process.env.RESEND_API_KEY!)
const BASE_URL = process.env.NEXT_PUBLIC_BASE_URL || 'http://localhost:3000'
const EMAIL_FROM = process.env.EMAIL_FROM || 'ferramentaria@noreply.com'
const EMAIL_ADMIN = process.env.EMAIL_ADMIN || ''

function templateBase(titulo: string, conteudo: string): string {
  return `<!DOCTYPE html><html lang="pt-BR">
  <body style="background:#f3f4f6;font-family:Arial,sans-serif;">
    <table width="100%" ...>
      <!-- Header vermelho SCiTec #C41E3A -->
      <tr><td style="background:#C41E3A;padding:24px 32px;">
        <h1 style="color:#fff;">🔧 Ferramentaria OS</h1>
        <p style="color:#fecdd3;">${titulo}</p>
      </td></tr>
      <!-- Conteúdo -->
      <tr><td style="padding:32px;">${conteudo}</td></tr>
      <!-- Rodapé -->
      <tr><td style="background:#f9fafb;...">E-mail automático...</td></tr>
    </table>
  </body></html>`
}

// 5 funções exportadas: enviarConfirmacaoPedido, enviarNotificacaoAdmin,
// enviarNotificacaoStatus, enviarNotificacaoAdminStatus, enviarNotificacaoAdminEdicao
// Todas usam resend.emails.send({ from, to, subject, html: templateBase(...) })
```

app/api/pedidos/route.ts — Criar e listar pedidos

app/api/pedidos/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import { supabaseServer } from '@/lib/supabase-server'
import { enviarConfirmacaoPedido, enviarNotificacaoAdmin } from '@/lib/email'
import { Pedido } from '@/types'

const schemaCriar = z.object({
  setor: z.string().min(1),
  solicitante: z.string().min(1),
  email_contato: z.string().email(),
  telefone: z.string().optional(),
  tipo_servico: z.string().min(1),
  descricao: z.string().min(10),
  urgencia: z.enum(['normal', 'alta', 'urgente']),
  prazo_desejado: z.string().optional(),
})

// GET — lista pedidos com filtros opcionais (status, urgencia, tipo_servico, busca)
export async function GET(request: NextRequest) { ... }

// POST — cria pedido, registra histórico, envia 2 e-mails
export async function POST(request: NextRequest) {
  // 1. Validar com zod
  // 2. INSERT na tabela pedidos
  // 3. INSERT historico_status (status inicial: 'recebido')
  // 4. enviarConfirmacaoPedido(pedido) — para o solicitante
  // 5. enviarNotificacaoAdmin(pedido)  — para o admin
  // 6. Retornar { pedido } com status 201
}
```

app/api/pedidos/[id]/status/route.ts — Mudar status (requer login admin)

app/api/pedidos/[id]/status/route.ts

```
// PATCH — verificar cookie JWT → buscar pedido → UPDATE status →
//         INSERT historico_status → enviarNotificacaoStatus (solicitante) →
//         enviarNotificacaoAdminStatus (admin)

export async function PATCH(request, { params }) {
  // 1. Verificar cookie 'ferramentaria_token' → verificarToken()
  // 2. Validar body: { status, observacao? }
  // 3. Buscar pedido atual no Supabase
  // 4. UPDATE pedidos SET status = statusNovo
  // 5. INSERT historico_status
  // 6. Enviar e-mails (try/catch independente para não bloquear)
  // 7. Retornar { pedido, historico }
}
```

app/api/pedidos/[id]/editar-solicitante/route.ts — Edição pública via token

app/api/pedidos/[id]/editar-solicitante/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import { supabaseServer } from '@/lib/supabase-server'
import { enviarNotificacaoAdminEdicao } from '@/lib/email'

const schemaEdicao = z.object({
  token: z.string().min(1),
  descricao: z.string().min(10).optional(),
  telefone: z.string().optional(),
  prazo_desejado: z.string().nullable().optional(),
})

export async function PATCH(request, { params }) {
  // 1. Validar body com zod (inclui token de acompanhamento)
  // 2. Buscar pedido por id + token_acompanhamento (autenticação por token)
  // 3. Rejeitar se status é 'concluido' ou 'cancelado'
  // 4. Detectar apenas os campos que mudaram
  // 5. UPDATE pedidos com os campos alterados
  // 6. INSERT historico_status com observação dos campos alterados
  // 7. enviarNotificacaoAdminEdicao(pedido, camposAlterados)
  // 8. Retornar { pedido: pedidoAtualizado }
}
```

app/api/auth/login/route.ts — Login admin

app/api/auth/login/route.ts

```
import bcrypt from 'bcryptjs'
import { criarToken } from '@/lib/auth'

// POST — valida email/senha → bcrypt.compare → criarToken JWT →
//        set-cookie 'ferramentaria_token' (httpOnly, 8h)
export async function POST(request) {
  const { email, senha } = await request.json()
  const usuario = await supabaseServer.from('usuarios')
    .select('*').eq('email', email).eq('ativo', true).single()

  const senhaValida = await bcrypt.compare(senha, usuario.senha_hash)
  if (!senhaValida) return 401

  const token = await criarToken({ id, email, nome, role })
  // Cookie: httpOnly, secure em prod, sameSite: 'lax', maxAge: 8h
  response.cookies.set('ferramentaria_token', token, { httpOnly: true, ... })
  return { ok: true, usuario }
}
```

app/acompanhar/[token]/page.tsx — Página de acompanhamento pública

app/acompanhar/[token]/page.tsx (Server Component)

```
// Server Component — busca dados no Supabase server-side
// Renderiza: hero do pedido (status + urgência), dados, timeline de histórico,
//            componente EditarPedido (client), anexos, comentários públicos

import { EditarPedido } from '@/components/pedido/EditarPedido'

export default async function AcompanharPage({ params }) {
  const pedido = await getPedidoPorToken(params.token)
  if (!pedido) notFound()

  const [historico, comentarios, anexos] = await Promise.all([
    getHistorico(pedido.id),
    getComentariosPublicos(pedido.id),
    getAnexos(pedido.id),
  ])

  return (
    <main>
      {/* hero com status */}
      {/* card de dados */}
      {/* timeline de histórico */}

      {/* Edição: só mostra se não está concluído/cancelado */}
      {pedido.status !== 'concluido' && pedido.status !== 'cancelado' && (
        <EditarPedido
          pedidoId={pedido.id}
          token={params.token}
          descricaoAtual={pedido.descricao}
          telefoneAtual={pedido.telefone || ''}
          prazoDesejadoAtual={pedido.prazo_desejado || ''}
        />
      )}

      {/* Anexos e Comentários */}
    </main>
  )
}
```

components/forms/FormularioPedido.tsx — Formulário de abertura de pedido

components/forms/FormularioPedido.tsx (Client Component)

```
'use client'

// Carrega tipos de serviço de /api/tipos-servico via useEffect
// Seções: 1. Identificação | 2. Detalhes do Serviço | 3. Prazo e Urgência

// Funcionalidade "Adicionar novo tipo":
// - Select tem opção value="__novo__" → ➕ Adicionar novo tipo...
// - Ao selecionar, exibe input de texto + botões Confirmar/Cancelar
// - Ao confirmar, o texto digitado vira form.tipo_servico
// - Mostra "Novo tipo: '...'" abaixo do select como confirmação

// Upload de anexos:
// - Estado: arquivos: File[]
// - Label estilizado como dropzone (click para selecionar, múltiplos)
// - Lista de arquivos selecionados com botão de remover
// - No submit: POST /api/pedidos → depois POST /api/pedidos/{id}/anexos para cada arquivo

// Após submit com sucesso: exibe card de confirmação com
// número do pedido (#XXXX) + link de acompanhamento
```

components/pedido/EditarPedido.tsx — Edição pelo solicitante

components/pedido/EditarPedido.tsx (Client Component)

```
'use client'

// Props: pedidoId, token, descricaoAtual, telefoneAtual, prazoDesejadoAtual
// Estado: aberto (bool), descricao, telefone, prazoDesejado, salvando, error, success

// Comportamento:
// - Quando aberto=false: exibe botão "Editar pedido" + texto explicativo
// - Quando aberto=true: exibe formulário com os 3 campos editáveis
// - Submit: PATCH /api/pedidos/{pedidoId}/editar-solicitante
//           body: { token, descricao, telefone, prazo_desejado }
// - Sucesso: fecha form, mostra "Pedido atualizado! A ferramentaria foi notificada por e-mail."
// - Cancelar: restaura valores originais (descricaoAtual, etc.)
```

tailwind.config.ts — Paleta SCiTec

tailwind.config.ts (trecho relevante)

```
theme: {
  extend: {
    colors: {
      brand: {
        50:  '#FFF1F2',
        100: '#FFE4E6',
        200: '#FECDD3',
        300: '#FDA4AF',
        400: '#FB7185',
        500: '#E0243A',
        600: '#C41E3A',   // ← cor principal SCiTec
        700: '#9F1828',
        800: '#7E1220',
        900: '#5C0D18',
      },
    },
  },
},
```

7. Fluxos Principais

Fluxo: Abertura de pedido

1. Colaborador acessa `/solicitar`
2. Preenche o formulário (FormularioPedido.tsx)
3. `POST /api/pedidos` → valida com zod → INSERT na tabela pedidos → INSERT historico_status
4. Se há arquivos: `POST /api/pedidos/{id}/anexos` para cada arquivo (upload para Supabase Storage)
5. E-mail de confirmação enviado ao solicitante (Resend)
6. E-mail de notificação enviado ao admin (Resend)
7. Formulário mostra número do pedido + link de acompanhamento único

Fluxo: Movimentação de card (admin)

1. Admin faz login em `/admin` → cookie JWT definido (8h)
2. No Kanban (`/admin/dashboard/kanban`), arrasta card para outra coluna
3. `PATCH /api/pedidos/{id}/status` com `{ status: 'em_andamento', observacao? }`
4. Middleware verifica cookie JWT antes de chegar na rota
5. UPDATE pedidos, INSERT historico_status
6. E-mail para solicitante notificando mudança de status
7. E-mail para admin confirmando a movimentação

Fluxo: Solicitante edita pedido

1. Solicitante acessa seu link `/acompanhar/{token}`
2. Se status ≠ concluído/cancelado, vê o componente EditarPedido
3. Clica em "Editar pedido", modifica campos desejados
4. `PATCH /api/pedidos/{id}/editar-solicitante` com token de acompanhamento no body
5. API verifica o token (sem necessidade de login), atualiza os campos, registra no histórico
6. E-mail enviado ao admin listando os campos alterados

8. Deploy e Hospedagem

Infraestrutura atual

HOSPEDAGEM
Vercel (Hobby — gratuito)

REPOSITÓRIO
GitHub (HeberPython/ferramentaria-os)

BANCO DE DADOS
Supabase Free Tier

E-MAIL
Resend Free (3k e-mails/mês)

Processo de deploy

1. Código alterado localmente com VS Code
2. `git add .` → `git commit -m "mensagem"` → `git push`
3. Vercel detecta o push automaticamente e inicia o build (`next build`)
4. Em ~2 minutos a nova versão está em produção

Comandos úteis em desenvolvimento

Terminal (pasta ferramentaria-os)

```
# Instalar dependências
npm install

# Iniciar servidor de desenvolvimento (http://localhost:3000)
npm run dev

# Fazer build de produção localmente
npm run build

# Verificar erros de TypeScript
npx tsc --noEmit

# Enviar para o GitHub (deploy automático Vercel)
git add .
git commit -m "descricao da mudanca"
git push
```

Migração para servidores da empresa (guia resumido)

1. **Servidor da empresa:** Precisa de Node.js 18+ instalado. Funciona em Windows Server ou Linux.
2. **Build:** Rodar `npm run build` no servidor e iniciar com `npm start` (porta 3000 por padrão).
3. **Variáveis de ambiente:** Criar arquivo `.env.production` ou configurar no ambiente do servidor (não usar .env.local em produção).
4. **Domínio interno:** Configurar proxy reverso (nginx ou IIS) apontando para localhost:3000.
5. **Banco de dados:** Pode continuar usando o Supabase cloud, ou migrar para PostgreSQL local usando `pg_dump` / `pg_restore`.
6. **E-mails:** Pode continuar usando Resend, ou configurar SMTP interno (substituir lib/email.ts por nodemailer).
7. **GitHub:** Remover a conta pessoal como owner e transferir o repositório para a organização SCiTec no GitHub.

Ferramentaria OS — SCiTec Laboratório · Documentação gerada em abril de 2026